Before concluding this section, let's create a Secret in the Kubernetes cluster with the Redis credentials required by Edge Service. In a real-world scenario, we should create a dedicated user for the application and grant limited privileges. For simplicity, we'll use the default account. Populate the Secret with the information returned by the previous `doctl` command:

```
$ kubectl create secret generic polar-redis-credentials \
    --from-literal=spring.redis.host=<redis_host> \
    --from-literal=spring.redis.port=<redis_port> \
    --from-literal=spring.redis.username=<redis_username> \
    --from-literal=spring.redis.password=<redis_password> \
    --from-literal=spring.redis.ssl=true
```

That's it for Redis. The following section will cover how to deploy RabbitMQ using a Kubernetes Operator.

B.4 Running RabbitMQ using a Kubernetes Operator

In the previous sections, we initialized and configured PostgreSQL and Redis servers that are offered and managed by the platform. We can't do the same for RabbitMQ because DigitalOcean doesn't have a RabbitMQ offering, similar to other cloud providers like Azure or GCP.

A popular and convenient way of deploying and managing services like RabbitMQ in a Kubernetes cluster is to use the *operator* pattern. Operators are "software extensions to Kubernetes that make use of custom resources to manage applications and their components" (https://kubernetes.io/docs/concepts/extend-kubernetes/operator).

Think about RabbitMQ. To use it in production, you'll need to configure it for high availability and resilience. Depending on the workload, you might